Edge cases (a template with the wrong variables) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the asymmetry
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the asymmetry

A `ChatPromptTemplate` knows, from parsing its own template string, exactly which variable names it needs. When `.invoke(...)` is called, it checks the dictionary you passed against that known set of names *before* doing anything else — including before any network call.

This means a missing variable fails fast, locally, with a `KeyError`-style message naming the exact missing variable — not a vague downstream failure. An *extra* variable your dictionary provides but the template doesn't use behaves differently: the template only looks for the keys it needs; an extra key that isn't referenced in the template string is silently ignored, not an error. This asymmetry matters — a missing variable is loud and cheap to catch; an extra, unused one is silent, which means a typo'd variable name you *meant* to use, but that doesn't match the template, won't be caught this way — it'll just be silently dropped.

Hint 2 — The plan, and almost the whole thing

```
prompt = ChatPromptTemplate.from_template("Answer: {question}")

try:
    prompt.invoke({})
except KeyError as e:
    print(f"Missing variable -> KeyError: {e}")

result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
print(f"Extra variable -> no error, result: {result}")
```

Notice the second call is not wrapped in `try/except` at all — because it's expected to succeed. If you find it doesn't, that's worth writing down as a real finding, not something to silently "fix" by assuming your first guess was right. Write both test calls yourself, then compare against the Solution.

Full solution: Show me the solution